

Lab 11: Micro labs de Google Cloud con Terraform

Diagramas de arquitectura, implementación con Terraform, RBAC y seguridad DevOps con Workload Identity Federation sobre GitLab CI, alineados con el Google Cloud Architecture Framework.

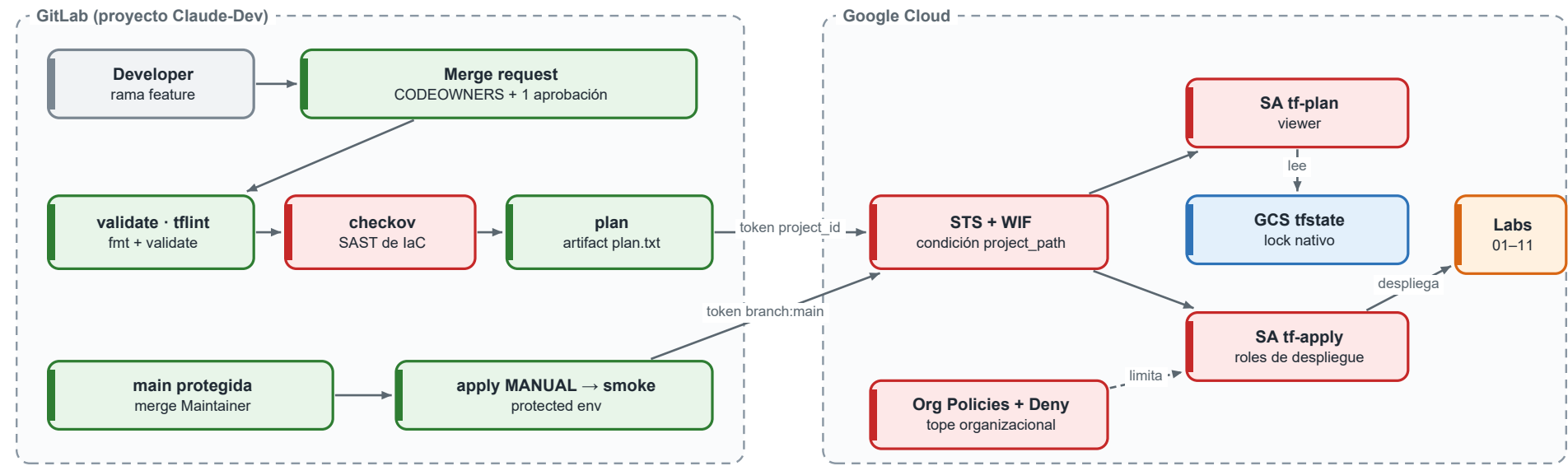
#	Micro lab	Servicios principales	Equivalente AWS (lab10_terraform_aws)
00	Bootstrap	GCS state, WIF, SAs, Artifact Registry, Budget	S3 + OIDC (00)
01	API Gateway + functions	API Gateway, Cloud Run functions, Firestore, API keys	API GW + Lambda (01)
02	Pub/Sub event-driven	Pub/Sub schema/push/DLQ/filtros, BigQuery sub, Scheduler	EventBridge (03)
03	Chatbot Vertex AI	Vertex AI Gemini, Firestore TTL	Bedrock (04)
04	Pipeline IA con Workflows	Eventarc, Workflows, Vision, Natural Language, Gemini, BigQuery	Step Functions (05)
05	Web de 3 niveles	LB global, Cloud Armor, MIG, Cloud SQL HA	ALB + RDS (06)
06	GKE Autopilot	GKE Autopilot, Workload Identity, HPA, PDB, NetworkPolicy	— (nuevo)
07	SRE observabilidad	Cloud Monitoring SLO, burn rate, uptime	CloudWatch SLO (07)
08	BigQuery analítica	BigQuery capas, partición, vista autorizada, scheduled query	— (nuevo)
09	Migración DMS	Database Migration Service, Cloud SQL	MGN (09)
10	Gobernanza de organización	Carpetas, Org Policies, tags, firewall jerárquico	StackSets + SCP (10)
11	Seguridad y gobernanza	Audit logs, sinks, IAM Deny, RBAC, break-glass	CloudTrail + GuardDuty (11)

Código fuente: [lab11_terraform_gcp/](#) (módulos en [modules/](#), labs en [microlabs/NN-nombre/](#)). Cada lab incluye README, terraform.tfvars.example, ci.yml y scripts/smoke-test.sh. Todos pasan terraform validate (Terraform 1.13 · google 7.x); no se han desplegado en un proyecto real.

Parte 1 · RBAC y seguridad DevOps en Google Cloud

Dos planos que se refuerzan: quién puede cambiar el código (roles de GitLab, ramas y entornos protegidos) y con qué identidad se despliega (service accounts impersonadas con Workload Identity Federation, nunca llaves JSON). Las Organization Policies y las IAM Deny Policies ponen un techo que ningún rol puede superar.

1.1 Flujo GitOps con Workload Identity Federation



■ Externo / usuario ■ Red ■ Cómputo ■ Datos / storage ■ Integración / API ■ IA / ML ■ Seguridad / identidad ■ Operación / observabilidad

El provider de WIF mapea los claims del token (project_id, ref_type, ref) a atributos. La SA de apply solo acepta el principalSet con deploy_ref = <project_id>:branch:main: una rama feature no puede impersonarla.

1.2 Matriz RBAC (GitLab ↔ Google Cloud)

Rol	GitLab (proyecto)	Google Cloud (grupo → roles)	Puede hacer	No puede hacer
Developer	Developer: ramas, MR, ver plan	group:devs → run.developer, datastore.user, logging/monitoring.viewer, rol custom deployer	Desplegar y depurar funciones y servicios	Administrar IAM; crear llaves de SA (deny); merge a main
SRE / Platform	Maintainer: merge, apply/destroy	group:sre → monitoring.editor, logging.viewer, compute.viewer, run.viewer	SLOs, alertas, runbooks, operar pipelines	Desactivar auditoría (deny opcional)
Security	Owner del grupo; CODEOWNER de labs 10/11	Org: orgpolicy.policyAdmin, iam.denyAdmin, securitycenter.admin	Org Políticas, deny policies, SCC, VPC-SC	Desplegar workloads
Auditor	Reporter	group:auditoria → iam.securityReviewer, logging.privateLogViewer, cloudasset.viewer	Leer IAM, logs (incl. Data Access) e inventario	Cualquier escritura
FinOps	Reporter	group:finops → recommender.viewer, billing.projectManager	Costos, recomendaciones de rightsizing	Cambios de infraestructura
CI plan	Job plan (cualquier rama/MR)	SA tf-plan ← principalSet attribute.project_id	Leer el proyecto y el estado	Modificar recursos
CI apply	Job apply (main, manual, protected env)	SA tf-apply ← principalSet attribute.deploy_ref ...:branch:main	Aplicar planes aprobados	Lo que niegan las Org/Deny policies
Break-glass	N/A (fuera del pipeline)	user:oncall → roles/owner con condición request.time < fecha	Recuperación de emergencia	Seguir teniendo acceso después de la fecha

Asigna roles a grupos de Cloud Identity / Google Workspace, nunca a usuarios individuales; usa IAM Conditions para acotar por recurso o por tiempo y revisa periódicamente con IAM Recommender (permisos no usados).

1.3 Capas de guardrails (defensa en profundidad)

Una acción solo se permite si ninguna Org Policy ni Deny Policy la bloquea y algún binding de IAM la concede.

Capa (de afuera hacia adentro)	Dónde se define	Qué limita	Micro lab
1. Organization Policies	google_org_policy_policy (carpeta/organización)	Configuraciones prohibidas para TODOS: llaves de SA, IPs externas, ubicaciones, SQL público	10
2. IAM Deny Policies	google_iam_deny_policy	Permisos denegados aunque un rol los otorgue (incluso a Owner), con excepciones explícitas	11
3. IAM allow (RBAC)	roles predefinidos + custom role + bindings por grupo	Lo que cada función necesita: developer, sre, auditor, finops	11
4. IAM Conditions	condition { expression }	Acceso acotado por recurso (Firestore de una base) o por tiempo (break-glass)	01 / 03 / 11
5. Políticas de recurso	IAM de bucket / dataset / función / topic	Quién usa cada recurso: invoker explícito, vista autorizada, agente de servicio	01–08
6. Red	Firewall jerárquico + VPC deny-all + PSA + VPC-SC	Sin IPs públicas; IAP; datos privados; perímetro de exfiltración	05 / 06 / 09 / 10 / 11

1.4 Controles DevSecOps del pipeline

Control	Implementación en lab11	Dónde
Sin secretos de larga duración	Workload Identity Federation: id_token → STS → impersonación; Org Policy bloquea llaves	.gitlab-ci.yml (.gcp_auth), labs 00 y 10
Separación de funciones	SA plan (viewer) ≠ SA apply (solo deploy_ref main); apply manual	lab 00, ci.yml de cada lab
Revisión obligatoria	main protegida, approvals, CODEOWNERS en modules/ y labs 10/11	Settings → Repository / Merge requests
Escaneo de IaC	fmt, validate, tfint, checkov (JUnit en el MR)	Etapas validate y security
Plan inmutable	apply usa el plan.tfplan del job plan	.tf_plan → .tf_apply
Verificación post-deploy	smoke test automático con outputs.json después del apply	Etapas smoke
Concurrencia segura	resource_group por lab + lock nativo de GCS	.tf_apply / .tf_destroy
Estado protegido	GCS versionado, soft delete, UBLA, PAP; acceso solo de SAs de CI	lab 00
Mínimo privilegio en runtime	Una SA por carga; IAM condicionado; invoker explícito	módulos service-account y cloud-function
Trazabilidad	default_labels (project, owner, microlab, cost_center) + audit logs	versions.tf, lab 11
Detección continua	Alertas de logs, SCC, burn rate, uptime	labs 07 y 11
Costo como control	Budgets, max_instances, require_partition_filter, destroy manual	labs 00, 01, 03, 08

1.5 Variables a configurar en GitLab

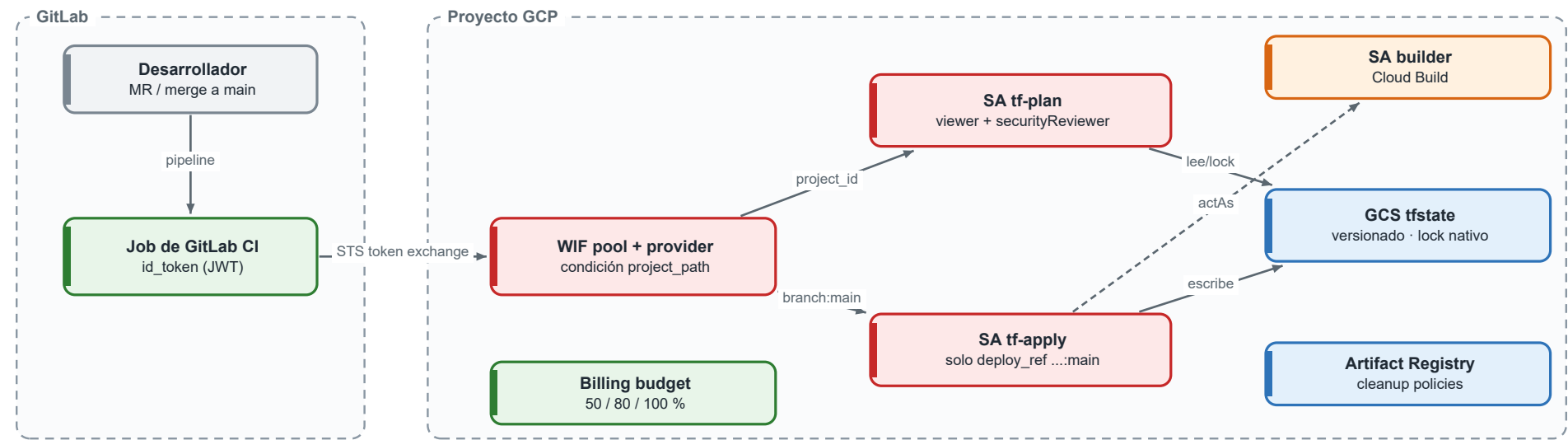
Variable	Tipo	Protegida	Enmascarada	Valor / origen
GCP_PROJECT_ID	Variable	No	No	ID del proyecto
TF_STATE_BUCKET	Variable	No	No	output tf_state_bucket (lab 00)
GCP_WIF_PROVIDER	Variable	No	No	output wif_provider
GCP_PLAN_SA	Variable	No	Sí	output plan_service_account
GCP_APPLY_SA	Variable	Sí	Sí	output apply_service_account
GCP01_TFVARS ... GCP11_TFVARS	File	Sí	No	terraform.tfvars de cada lab
DESTROY_LAB	Variable (al ejecutar)	No	No	Nombre del lab a destruir

Parte 2 · Arquitecturas e implementación por micro lab

Cada lab muestra el diagrama (colores por categoría; fronteras punteadas = organización / VPC / servicio), la tabla de implementación con Terraform y los datos de operación.

Micro lab 00 · Bootstrap: estado remoto, Workload Identity Federation y presupuesto

Prepara el proyecto para desplegar desde GitLab sin llaves JSON: estado en GCS, federación OIDC, SAs separadas de plan y apply, builder de Cloud Build y presupuesto.



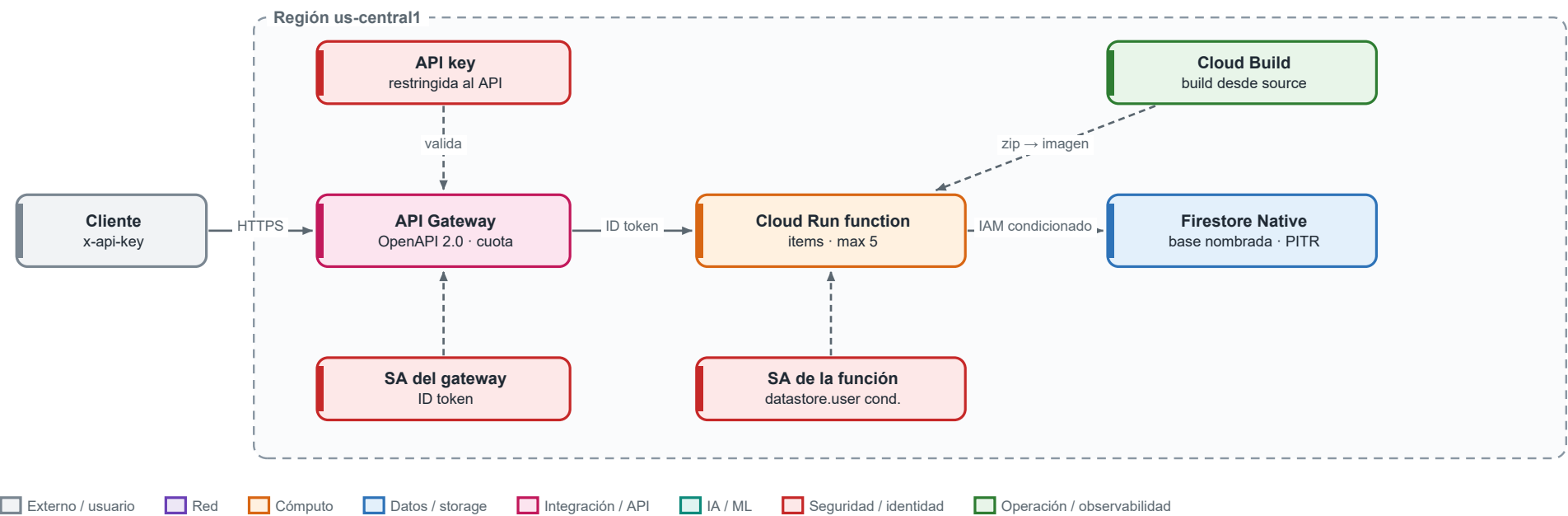
Externo / usuario Red Computo Datos / storage Integración / API IA / ML Seguridad / identidad Operación / observabilidad

Componente	Recurso / módulo Terraform	Configuración clave (parámetros)
Estado	module.state_bucket → modules/gcs-secure-bucket	force_destroy = false; versionado; soft delete 30 d; UBLA; public access prevention
Federación	google_iam_workload_identity_pool(_provider)	issuer https://gitlab.com; attribute_mapping (project_id, ref, deploy_ref); attribute_condition project_path
SA plan	module.plan_sa + google_service_account_iam_member.plan_wif	principalSet .../attribute.project_id/<id>; roles viewer + securityReviewer
SA apply	module.apply_sa + apply_wif	principalSet .../attribute.deploy_ref/<id>;:branch:main; apply_roles
Builder	module.builder_sa	cloudbuild.builds.builder, logWriter, artifactregistry.writer
Imágenes	google_artifact_registry_repository	keep 10 versiones; borra untagged > 7 días
Costo	google_billing_budget (count)	billing_account; monthly_budget; notificación por email
Variables GitLab	TF_STATE_BUCKET, GCP_WIF_PROVIDER, GCP_PLAN_SA, GCP_APPLY_SA (protegida), GCP_PROJECT_ID	
Storage mínimo	GCS < 1 MB por lab	

Architecture Framework	SEC: sin llaves, apply solo en main · OPS: GitOps · COST: budget
Prueba automatizada	bash scripts/lab.sh test 00-platform-bootstrap-gcs-wif → bucket seguro, provider ACTIVE, SA apply sin llaves y solo desde main

Micro lab 01 · API serverless: API Gateway + Cloud Run functions + Firestore

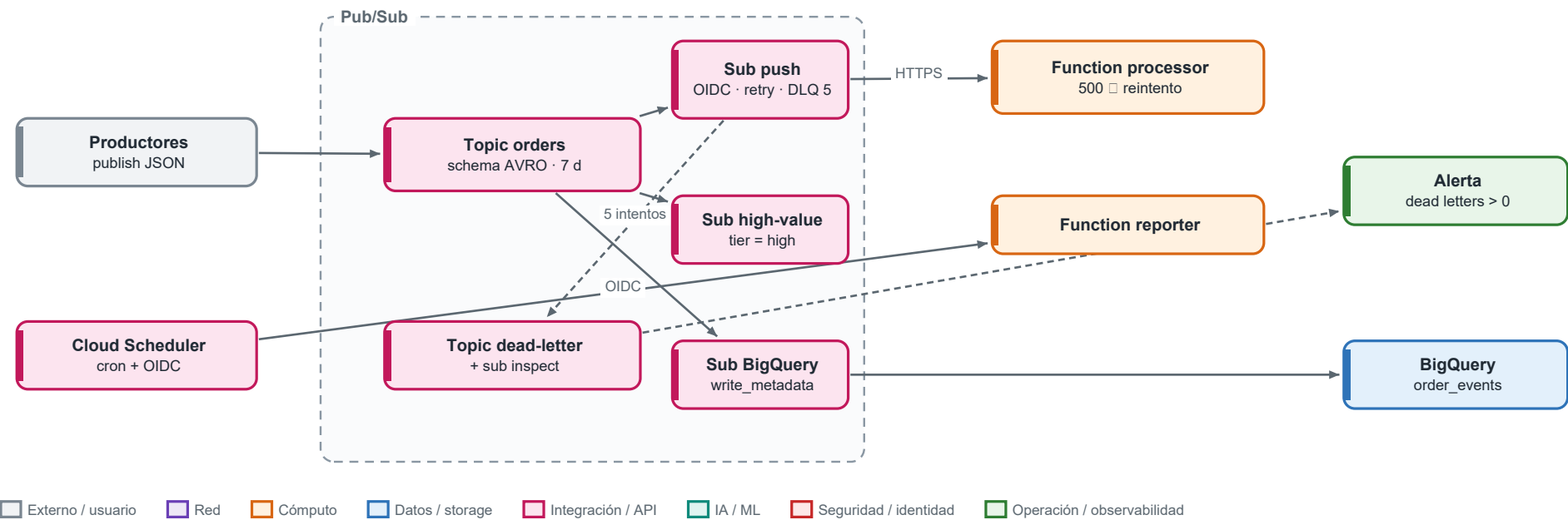
API REST con contrato OpenAPI, API keys con cuota por consumidor, backend privado (solo el gateway lo invoca) e IAM condicionado a una base de Firestore.



Componente	Recurso / módulo Terraform	Configuración clave (parámetros)
Contrato	openapi.yaml.tpl → base64encode(templatefile())	x-google-backend (APPEND_PATH), securityDefinitions api_key, x-google-management (quota_per_minute)
Gateway	google_api_gateway_api / _api_config / _gateway (google-beta)	api_config_id_prefix + create_before_destroy; backend_config con el SA del gateway
API key	google_apikeys_key + google_project_service.managed_api	restrictions.api_targets = servicio administrado del API
Función	module.items_fn → modules/cloud-function	python313; invoker_members = [SA del gateway] → llamada directa 403
Datos	google_firestore_database	FIRESTORE_NATIVE; PITR; IAM condition resource.name.startsWith(base)
Fallas (lab 07)	var.enable_fault_injection	Header x-fault-injection → 500 solo si está habilitado
Variables GitLab	GCP01_TFVARS (File)	
Storage mínimo	Firestore por operación; zips con expiración de 30 días	
Architecture Framework	SEC: key + cuota + backend privado · REL: PITR · COST: free tier	
Prueba automatizada	bash scripts/lab.sh test 01-serverless-apigw-functions-firestore → 401/400, 403 backend privado, CRUD 201→200→204→404, 429 por cuota	

Micro lab 02 · Orientado a eventos con Pub/Sub: schema, push OIDC, DLQ, filtros y BigQuery

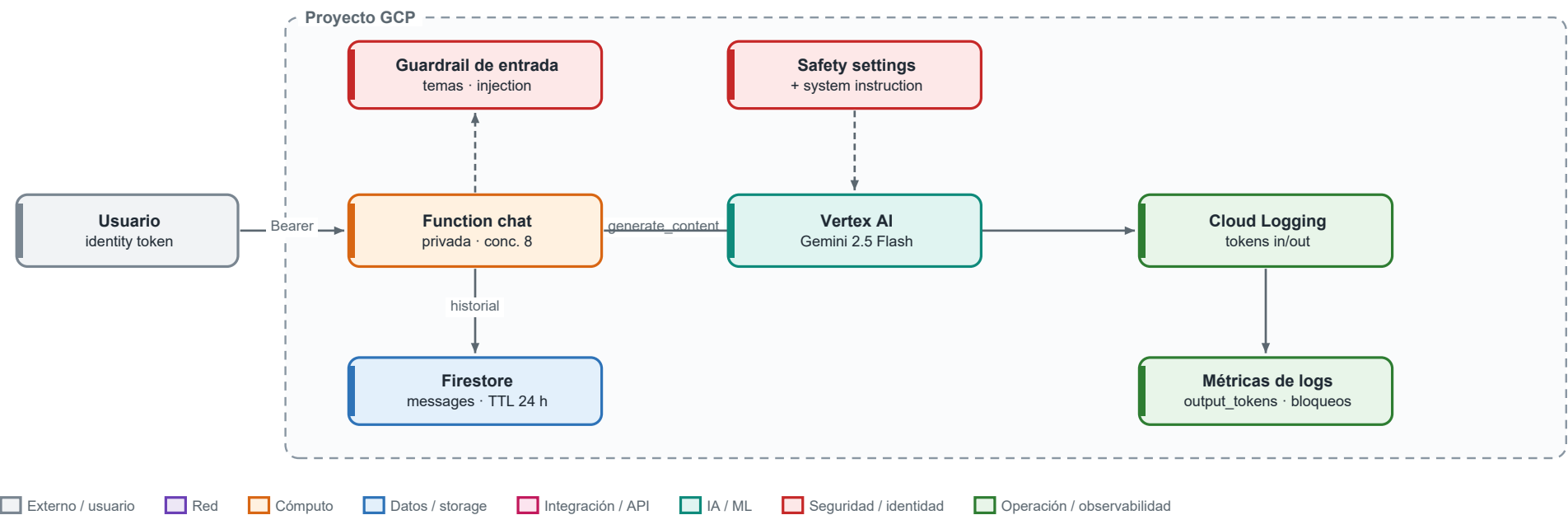
Contrato AVRO validado al publicar, push autenticado con reintentos y dead letter, suscripción filtrada, auditoría directa en BigQuery y Cloud Scheduler.



Componente	Recurso / módulo Terraform	Configuración clave (parámetros)
Contrato	google_pubsub_schema + schemas/order.avsc	AVRO con encoding JSON; uniones como {"double": 250}
Topic	google_pubsub_topic.orders	message_retention_duration 7 d → seek para replay
Push + DLQ	google_pubsub_subscription.processor	oidc_token (SA push); retry 10-60 s; dead_letter_policy max_delivery_attempts = 5
Agente Pub/Sub	topic/subscription IAM + tokenCreator	publisher en DLQ, subscriber en la sub, firma OIDC
Filtro	google_pubsub_subscription.high_value	filter = attributes.tier = "high"
Auditoría	bigquery_config + google_bigquery_table.events	write_metadata; partición diaria; audit_retention_days
Programación	google_cloud_scheduler_job.report	report_schedule, schedule_timezone, oidc_token
Variables GitLab	GCP02_TFVARS (File)	
Storage mínimo	Retención Pub/Sub 7/14 d; BigQuery 30 d	
Architecture Framework	REL: retry + DLQ + replay · SEC: push OIDC · COST: sin código para auditar	
Prueba automatizada	bash scripts/lab.sh test 02-events-pubsub-scheduler → schema rechaza inválidos, filtro, DLQ tras 5 intentos, BigQuery, Scheduler	

Micro lab 03 · Chatbot de IA generativa con Vertex AI Gemini

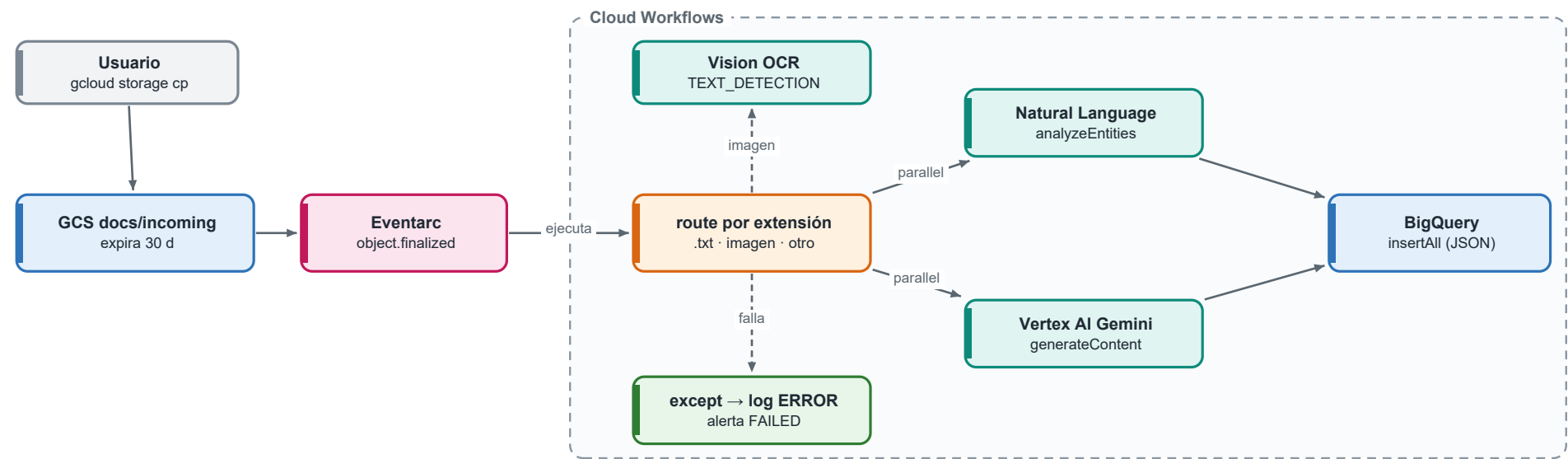
Chatbot privado con Gemini (SDK google-genai), historial en Firestore con TTL nativo, guardrails en capas y métricas de tokens en Logging.



Componente	Recurso / módulo Terraform	Configuración clave (parámetros)
Código	src/chat/main.py	genai.Client(vertexai=True); historial por email#session; finish_reason / prompt_feedback
Historial	google_firestore_database + _index + _field (ttl_config)	Índice session_id + ts; TTL sobre expires_at
Identidad	module.chat_sa	aiplatform.user + datastore.user condicionado a la base
Función	module.chat_fn	512 Mi, 1 vCPU, concurrencia 8; invoker_members; max_instances = tope de gasto
Parámetros IA	model, vertex_location, system_instruction	max_output_tokens = 512; history_turns = 6; blocked_topics
Observabilidad	google_logging_metric x2	Distribución de output_tokens; conteo de guardrail_blocks
Variables GitLab	GCP03_TFVARS (File), TF_VAR_system_instruction	
Storage mínimo	Firestore con TTL 24 h	
Architecture Framework	SEC: función privada + 3 capas de guardrails · COST: tokens e instancias acotados	
Prueba automatizada	bash scripts/lab.sh test 03-genai-chatbot-vertex-gemini → 403 sin token, STOP + tokens, memoria, tema e injection bloqueados	

Micro lab 04 · Pipeline de IA para documentos con Cloud Workflows

Orquestar APIs de IA con conectores de Workflows, sin funciones propias: OCR, entidades y resumen en paralelo, con try/except y resultados en BigQuery.

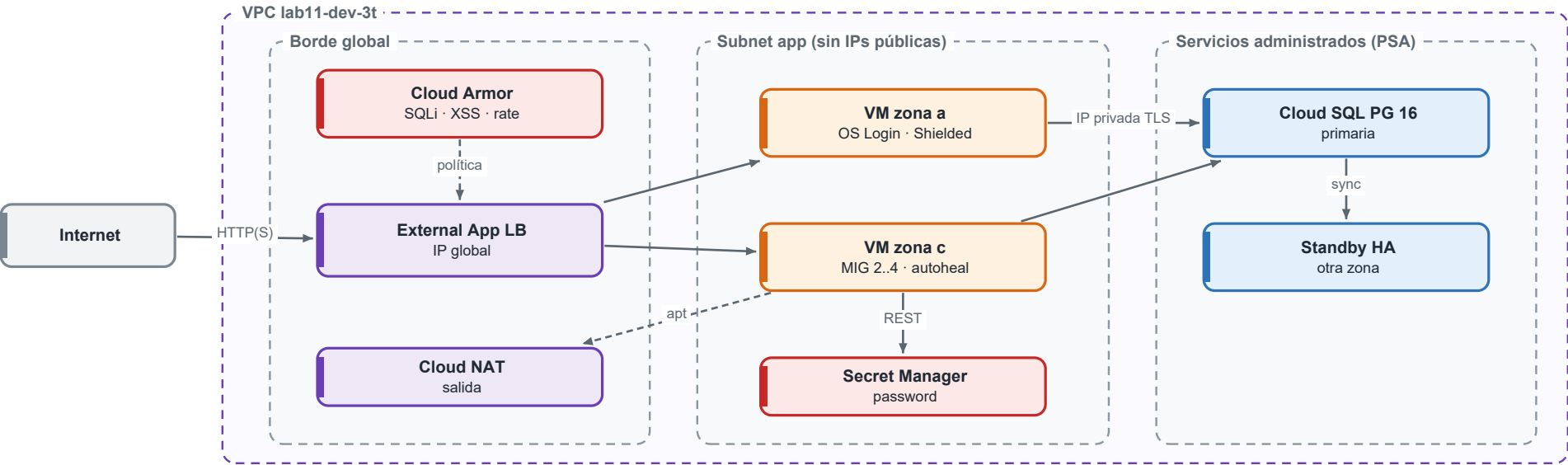


■ Externo / usuario ■ Red ■ Cómputo ■ Datos / storage ■ Integración / API ■ IA / ML ■ Seguridad / identidad ■ Operación / observabilidad

Componente	Recurso / módulo Terraform	Configuración clave (parámetros)
Definición	workflow.yaml → file()	Expresiones con '.' entre comillas; switch con next; parallel shared [entities, summary]
Workflow	google_workflows_workflow	user_env_vars (MODEL, VERTEX_LOCATION, LANGUAGE, DATASET, TABLE); call_log_level LOG_ERRORS_ONLY
Disparador	google_eventarc_trigger + IAM del agente GCS	type + bucket; SA con workflows.invoker y eventarc.eventReceiver
Identidad	module.workflow_sa	aiplatform.user, serviceUsageConsumer; objectViewer en el bucket; dataEditor en el dataset
Resultados	google_bigquery_table.documents	entities tipo JSON; partición diaria por processed_at
Alerta	google_monitoring_alert_policy.failed	finished_execution_count{status=FAILED} > 0
Variables GitLab	GCP04_TFVARS (File)	
Storage mínimo	GCS 30 d; 1 fila por documento	
Architecture Framework	OPS: historial visual por paso · COST: sin servidores · PERF: paralelismo	
Prueba automatizada	bash scripts/lab.sh test 04-genai-docs-workflows → SUCCEEDED con entidades + resumen; .docx FAILED; otros/ ignorado	

Micro lab 05 · Arquitectura web de 3 niveles: Load Balancer + MIG + Cloud SQL HA

LB global con Cloud Armor, MIG regional sin IPs públicas con autohealing, y Cloud SQL PostgreSQL HA solo por IP privada con TLS y Secret Manager.

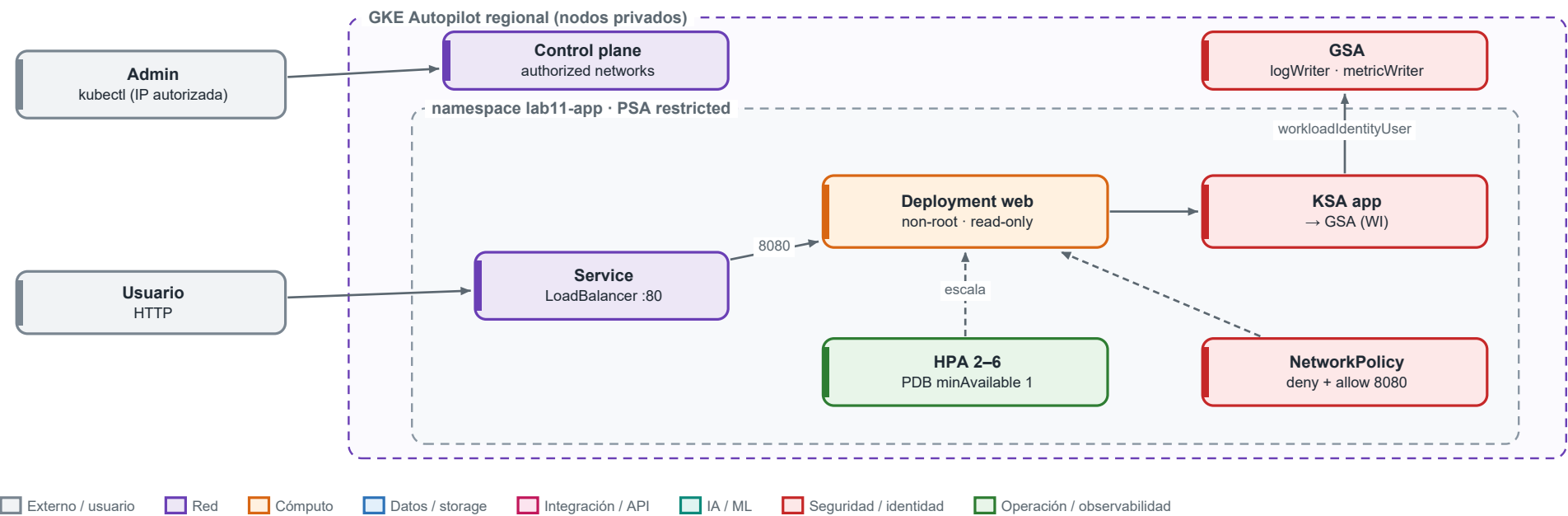


Externo / usuario Red Computo Datos / storage Integración / API IA / ML Seguridad / identidad Operación / observabilidad

Componente	Recurso / módulo Terraform	Configuración clave (parámetros)
Red	module.vpc	subnet app; Cloud NAT; Private Service Access; health checks 130.211/22 y 35.191/16; IAP
Datos	google_sql_database_instance.db	REGIONAL; ipv4_enabled = false; ssl_mode ENCRYPTED_ONLY; PITR; backups 7; Query Insights
Credenciales	random_password + google_secret_manager_secret	La VM la lee con el token del SA (sin llaves)
App	google_compute_instance_template + startup.sh.tpl	Debian 12; app Python stdlib (/ , /health, /db); Shielded VM; OS Login
Cómputo	region_instance_group_manager + region_autoscaler	autohealing 180 s; update PROACTIVE max_unavailable 0; CPU 60 %
WAF	google_compute_security_policy.waf	sqli-v33-stable, xss-v33-stable, rate_based_ban rate_limit_per_minute
Borde	backend_service / url_map / http(s) proxy / forwarding_rule	EXTERNAL_MANAGED; HTTPS con certificado administrado si domain != null
Variables GitLab	GCP05_TFVARS (File); timeout 1 h	
Storage mínimo	10 GB por VM; Cloud SQL 10 GB SSD con autoresize	
Architecture Framework	REL: MIG regional + SQL HA · SEC: Armor, sin IPs públicas · COST: ~USD 120/mes	
Prueba automatizada	bash scripts/lab.sh test 05-web3tier-lb-mig-cloudsql → 2 zonas sanas, /db TLS, SQL REGIONAL + PITR, Armor SQLi/XSS; CHAOS=1	

Micro lab 06 · GKE Autopilot privado con workload endurecido

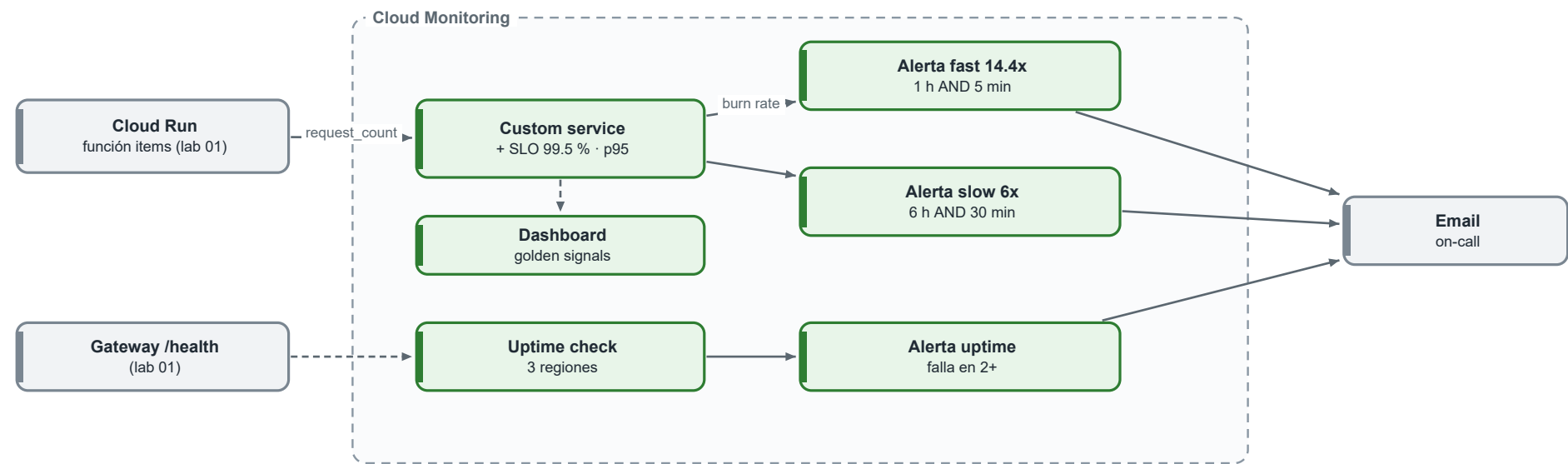
Kubernetes administrado con seguridad por defecto: nodos privados, Workload Identity, Pod Security restricted, NetworkPolicy, HPA y PDB, todo en Terraform.



Componente	Recurso / módulo Terraform	Configuración clave (parámetros)
Red	module.vpc (secondary_ranges pods/services)	nodes_cidr, pods_cidr, services_cidr; Cloud NAT para imágenes externas
Cluster	google_container_cluster	enable_autopilot; enable_private_nodes; master_authorized_networks; release REGULAR; maintenance window
Identidad	google_service_account_iam_member.workload_identity	<project>.svc.id.goog[lab11-app/app] → GSA
Provider k8s	provider "kubernetes" (token de google_client_config)	Despliega en el mismo apply (repite apply si el cluster es nuevo)
Workload	kubernetes_deployment_v1	runAsNonRoot, seccomp RuntimeDefault, drop ALL, readOnlyRootFilesystem, probes, topology spread
Resiliencia	HPA v2 + PDB + Service LoadBalancer	min/max_replicas; CPU 60 %; minAvailable 1
Zero-trust	kubernetes_network_policy_v1 x2	default-deny-ingress + allow-web-8080
Variables GitLab	GCP06_TFVARS (File)	
Storage mínimo	emptyDir /tmp; sin volúmenes persistentes	
Architecture Framework	SEC: PSA + WI + nodos privados · REL: regional + PDB · COST: ~USD 95/mes	
Prueba automatizada	bash scripts/lab.sh test 06-k8s-gke-autopilot → nodos privados, WI, pod privilegiado rechazado, LB 200, PDB/NetPol; LOAD=1	

Micro lab 07 · SRE con Cloud Monitoring: SLOs, burn rate, uptime y dashboard

SLOs request-based sobre la función del lab 01 y alertas por consumo del presupuesto de error (multi-window, multi-burn-rate), con uptime checks y señales doradas.

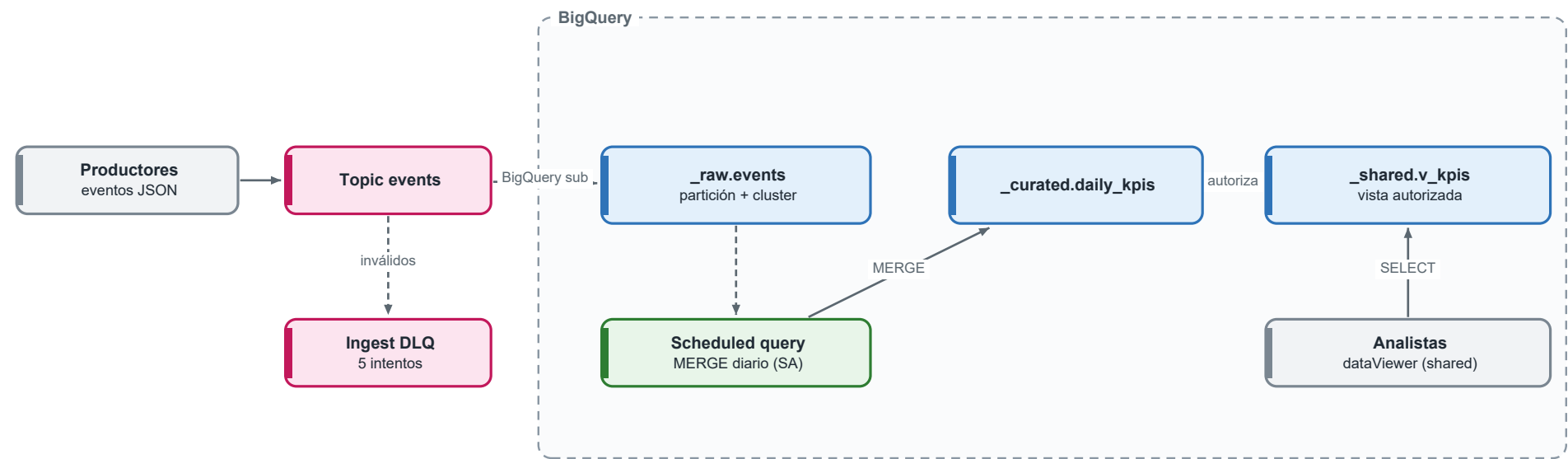


Externo / usuario Red Computo Datos / storage Integración / API IA / ML Seguridad / identidad Operación / observabilidad

Componente	Recurso / módulo Terraform	Configuración clave (parámetros)
Integración	data.terraform_remote_state.lab01 (gcs)	lab01_state_bucket → servicio y host sin copiarlos a mano
Servicio	google_monitoring_custom_service	Agrupar los SLOs del API
SLO disponibilidad	google_monitoring_slo.availability	request_based_sli.good_total_ratio: bad = 5xx, total = request_count; availability_goal
SLO latencia	google_monitoring_slo.latency	distribution_cut request_latencies < latency_threshold_ms; latency_goal
Burn rate	google_monitoring_alert_policy.burn (for_each)	select_slo_burn_rate(slo, ventana) > tasa; combiner AND; severidad
Caja negra	google_monitoring_uptime_check_config + alerta	SSL validado; REDUCE_COUNT_FALSE > 1
Dashboard	google_monitoring_dashboard + dashboard.json.tpl	Tráfico por clase, p50/p95/p99, instancias, burn rate, uptime
Variables GitLab	GCP07_TFVARS (File)	
Storage mínimo	Métricas retenidas 6 semanas sin costo	
Architecture Framework	OPS: SLOs + runbooks · REL: alertas por presupuesto de error	
Prueba automatizada	bash scripts/lab.sh test 07-sre-slo-monitoring → 2 SLOs, alertas; GENERATE_ERRORS=1 → burn rate 5 min > 14.4	

Micro lab 08 · Analítica con BigQuery: ingesta streaming, capas y vista autorizada

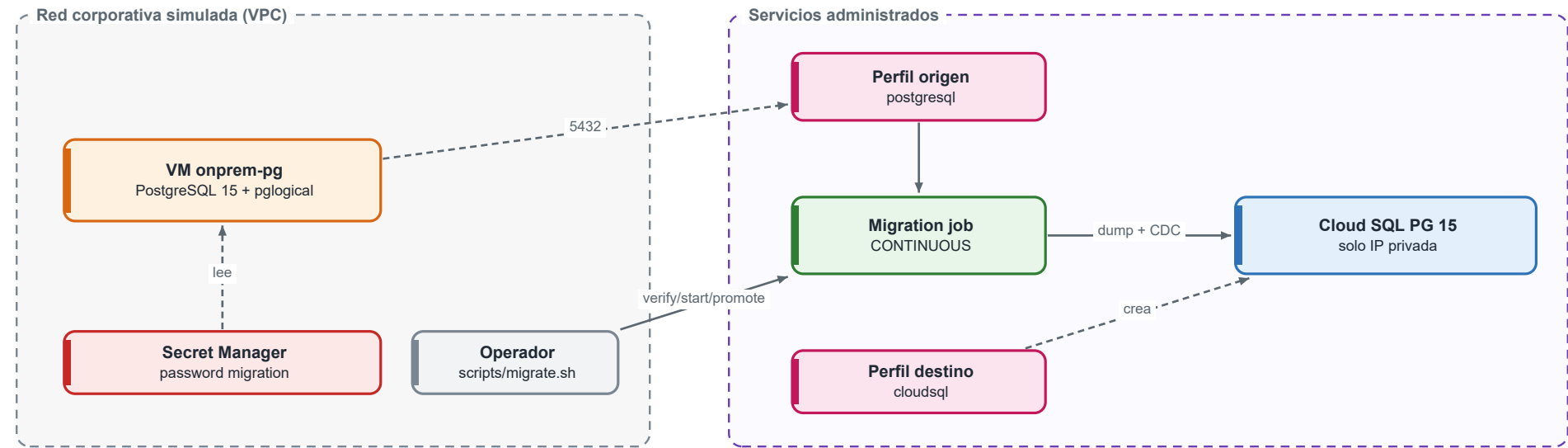
Data platform mínima: ingesta sin código desde Pub/Sub, partición + clustering con filtro obligatorio, MERGE programado y analistas solo sobre vistas sin PII.



Componente	Recurso / módulo Terraform	Configuración clave (parámetros)
Capas	google_bigquery_dataset x3	_raw (PII, expiración de particiones), _curated, _shared
Tabla cruda	google_bigquery_table.events + schemas/events.json	partición DAY event_ts; clustering country, event_type; require_partition_filter
Ingesta	google_pubsub_subscription.to_bigquery	use_table_schema; drop_unknown_fields; dead_letter_policy
Transformación	google_bigquery_data_transfer_config + sql/merge_daily_kpis.sql.tpl	scheduled_query; kpi_schedule; service_account_name dedicada
Acceso	google_bigquery_dataset_access.authorized_view	curated autoriza la vista; analistas sin acceso a raw/curated
Analistas	dataset IAM + jobUser	analyst_members
Variables GitLab	GCP08_TFVARS (File)	
Storage mínimo	Particiones crudas 90 d; KPIs en KB/día	
Architecture Framework	COST: filtro de partición + dry run · SEC: vista sin PII	
Prueba automatizada	bash scripts/lab.sh test 08-data-bigquery-analytics → 30 filas, consulta sin partición rechazada, DLQ, MERGE, vista sin PII	

Micro lab 09 · Migración con Database Migration Service (dump + CDC)

Migrar PostgreSQL a Cloud SQL con DMS continuo: origen on-prem simulado con pglogical, conectividad privada por VPC peering y cutover controlado (promote).

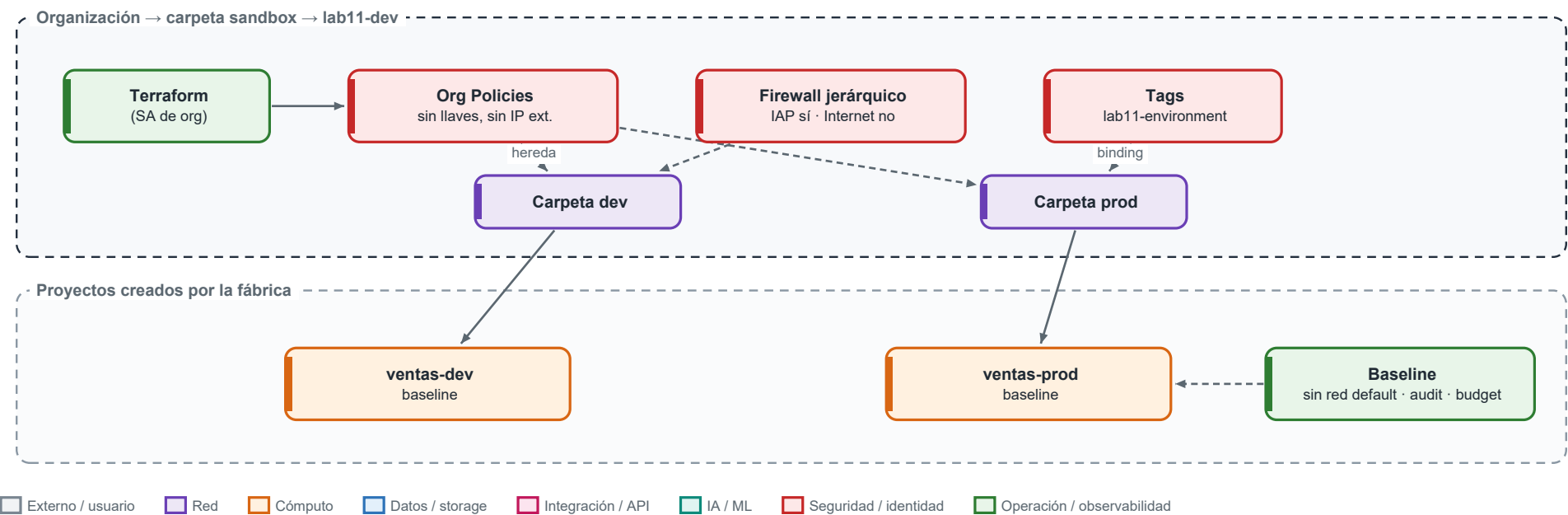


Externo / usuario Red Computo Datos / storage Integración / API IA / ML Seguridad / identidad Operación / observabilidad

Componente	Recurso / módulo Terraform	Configuración clave (parámetros)
Origen	google_compute_instance.source + source-startup.sh.tpl	wal_level logical; pglogical en todas las bases; usuario REPLICATION; tablas con PK
Red	module.vpc (PSA) + firewall allow-dms-5432	VPC peering para DMS y Cloud SQL; sin IPs públicas
Credenciales	random_password + Secret Manager	La VM lee su contraseña con el token del SA
Perfiles	google_database_migration_service_connection_profile x2	postgresql (host/puerto/usuario) y cloudsql.settings (POSTGRES_15, ip_config privada)
Job	google_database_migration_service_migration_job	type CONTINUOUS; vpc_peering_connectivity
Operación	scripts/migrate.sh	verify → start → status → promote (cutover) → delete
Variables GitLab	GCP09_TFVARS (File)	
Storage mínimo	VM 20 GB; Cloud SQL 20 GB SSD	
Architecture Framework	REL: CDC, RPO de segundos · SEC: todo privado · VMs: Migrate to Virtual Machines	
Prueba automatizada	bash scripts/lab.sh test 09-migration-dms → origen 500/5000 filas, verify; RUN_MIGRATION=1 → fase CDC	

Micro lab 10 · Gobernanza de organización: Org Policies, tags y fábrica de proyectos

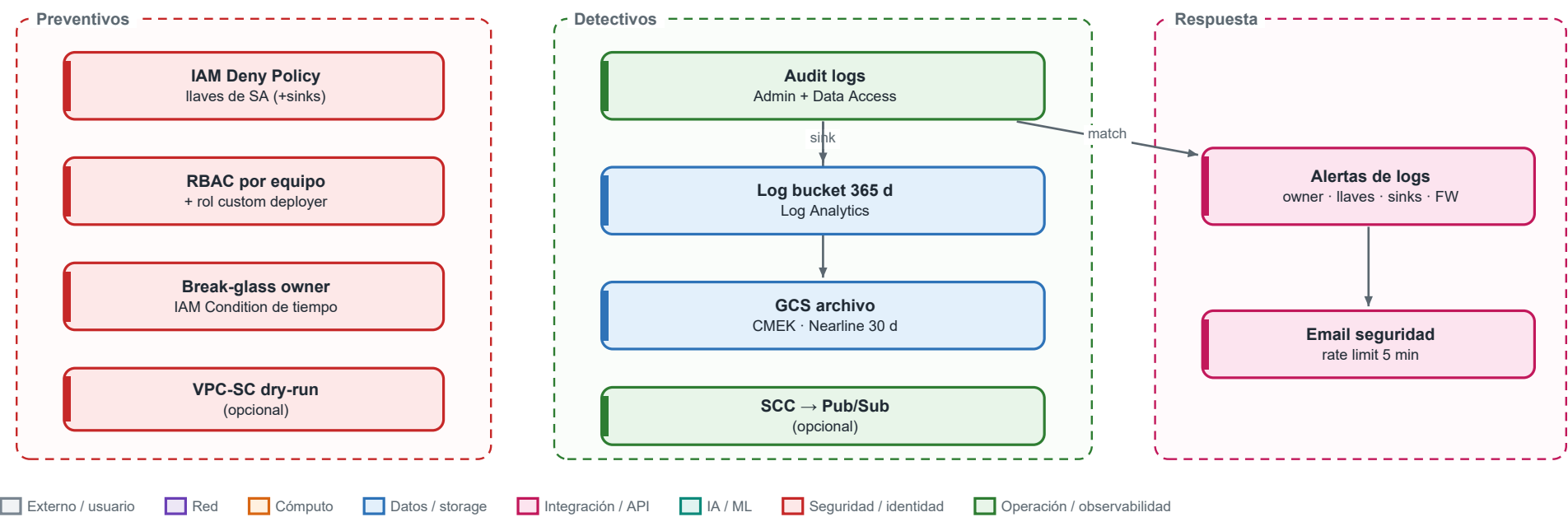
El equivalente de StackSets + SCPs: guardrails heredados por carpeta, firewall jerárquico, políticas condicionadas por tag y N proyectos con el mismo baseline.



Componente	Recurso / módulo Terraform	Configuración clave (parámetros)
Jerarquía	google_folder root + env (for_each)	parent_folder_id (sandbox) u organización
Guardrails	google_org_policy_policy (boolean)	disableServiceAccountKeyCreation, uniformBucketLevelAccess, publicAccessPrevention, requireOsLogin, restrictPublicIp
Listas	vmExternallpAccess deny_all; gcp.resourceLocations	allowed_locations = in:us-locations
Condicional	gcp.restrictServiceUsage + tags	resource.matchTag(org/lab11-environment, dev) → allow_all; resto deny
Red	google_compute_firewall_policy + rules + association	allow IAP 22/3389; deny 22/3389 desde 0.0.0.0/0
Fábrica	module.projects (modules/project-baseline)	auto_create_network = false; APIs; audit config; operadores; budget por proyecto
Variables GitLab	GCP10_TFVARS (File); SA con permisos de organización	
Storage mínimo	Ninguno	
Architecture Framework	SEC: guardrails preventivos heredados · OPS: baseline idéntico	
Prueba automatizada	bash scripts/lab.sh test 10-governance-org-policies → políticas efectivas por proyecto, sin red default, llave de SA bloqueada	

Micro lab 11 · Seguridad del proyecto: auditoría, IAM Deny, RBAC y alertas

Controles detectivos y preventivos: Data Access logs, archivo con CMEK, alertas sobre eventos de alto riesgo, deny policy, RBAC por equipo y break-glass con caducidad.



Componente	Recurso / módulo Terraform	Configuración clave (parámetros)
Auditoría	google_project_iam_audit_config (for_each)	ADMIN_READ, DATA_READ, DATA_WRITE en data_access_audit_services
Retención	google_logging_project_bucket_config + sinks	log_retention_days = 365; enable_analytics; lock_log_bucket (irreversible)
Archivo	module.kms + module.archive + sink GCS	CMEK; Nearline a 30 d; retention_days; writer_identity objectCreator
Alertas	google_monitoring_alert_policy.log_alerts	condition_matched_log; notification_rate_limit 5 min; auto_close 30 min
Deny	google_iam_deny_policy.guardrails	denied_principals public:all; exception_principals; protect_audit_sinks
RBAC	google_project_iam_custom_role + google_project_iam_member.rbac	developer, sre, auditor, finops (rbac_members)
Break-glass	google_project_iam_member.break_glass	condition request.time < timestamp(break_glass_until)
Variables GitLab	GCP11_TFVARS (File); aplicar primero	
Storage mínimo	Log bucket 365 d; GCS Nearline	
Architecture Framework	SEC: deny + RBAC + auditoría immutable · COST: Data Access selectivo	
Prueba automatizada	bash scripts/lab.sh test 11-security-iam-deny-rbac → auditoría y sinks, deny bloquea llaves, evento en el log bucket, RBAC	

Anexo · Matriz del Architecture Framework por micro lab

Resumen por pilar; el detalle está en la sección correspondiente de cada README.

Lab	Excelencia operativa	Seguridad	Confiabilidad	Rendimiento	Costos	Sostenibilidad
00	GitOps + estado versionado	WIF sin llaves	Soft delete del estado	N/A	Budget	Solo administrados
01	Contrato OpenAPI	Backend privado + cuota	PITR Firestore	Escala a cero	Free tier	Sin cómputo ocioso
02	Schema + replay	Push OIDC	Retry + DLQ	Filtro en broker	Auditoría sin código	Por evento
03	Métricas de tokens	Privada + guardrails	Tope de instancias	Concurrencia 8	Tokens acotados	Modelo flash
04	Historial por paso	SA mínima	try/except + alerta	Parallel	Sin servidores	Datos expiran
05	Rolling proactivo	Armor, sin IPs	MIG regional + SQL HA	LB global	HA opcional	Autoscaling
06	Workload en TF	PSA + WI + NetPol	PDB + regional	HPA	Pago por pod	Bin packing
07	SLOs + runbooks	Sin datos sensibles	Burn rate	SLO p95	Gratis	Menos ruido
08	SQL versionado	Vista sin PII	DLQ de ingesta	Partición + cluster	Filtro obligatorio	Menos bytes
09	verify + promote	Todo privado	CDC	Dump paralelo	DMS sin costo	Apagar origen
10	Baseline idéntico	Org Políticas	Aislamiento	N/A	Budget por proyecto	Ubicaciones
11	Log Analytics	Deny + RBAC	Doble destino	N/A	Data Access selectivo	Retención acotada

Orden recomendado: 00 → 11 → 01 → 02 → 03 → 04 → 07 → 08 → 05 → 06 → 09 → 10. Destruye los labs 05 (~USD 120/mes), 06 (~USD 95/mes) y 09 (~USD 2/día) al terminar cada sesión.